

UNIVERSITY OF LEEDS

COMP1850 Programming

Week 7, Session 2

Introduction to Debugging

Recap

Last time, we learned

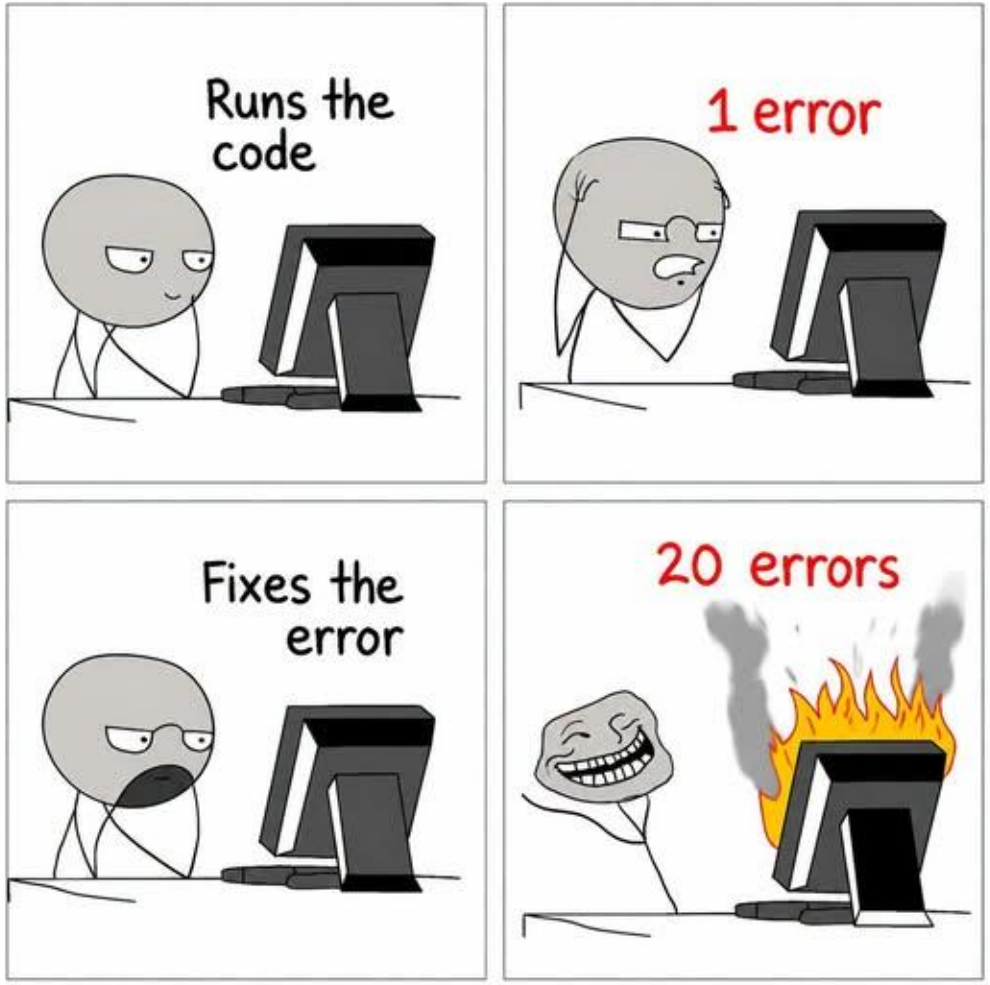
- How to identify equivalence classes and choose values from those classes that we can use in testing
- How to implement a unit test with `assert`
- How to use the `pytest` unit testing framework

UNIVERSITY OF LEEDS

Today's Aims

- Introduction to debugging
- Debugging approaches and tools in Python

Every programmer knows this feeling



Debugging is part of the journey, and every mistake is another step toward becoming a better developer

…. Tech Learning Zone

UNIVERSITY OF LEEDS

What is Debugging?

"The process of identifying, understanding and removing the **bugs** in a program"

- Debugging is a skill that you need to master over time
- With tips and tricks specific to different aspects of programming

UNIVERSITY OF LEEDS

The Nature of Bugs

- With many bugs, a program may run to completion but still behave incorrectly – e.g., produce wrong result from a calculation
- Some bugs will cause a program to crash (i.e., terminate abnormally), potentially before it has done anything useful
- Some bugs are vulnerabilities that can be exploited by an attacker to perform a malicious action with a program

(See https://en.wikipedia.org/wiki/Software_bug for a history of software bugs)

UNIVERSITY OF LEEDS

Program Crashes in Python

- Python compiles to bytecode which is then immediately executed by the Python Virtual Machine (PVM)
- The PVM can provide high-level information on crashes (type of error, line number where the problem was detected, etc...)

UNIVERSITY OF LEEDS

Beginning Tips for Debugging

- Bugs are unexpected behaviour
- Therefore, to identify them, you need to know precisely what the expected behaviour of your program is
- You can find clues from:
  - Error messages
  - Print statements
- The best way of capturing that expected behaviour is in the form of a comprehensive suite of automated tests

UNIVERSITY OF LEEDS

Know Your Errors

- Many problems in your Python program will lead to an exception
- Read the error description associated with this exception
- Common errors:
  - `SyntaxError` - error in syntax rather than a bug in run-time behaviour (e.g. missing parenthesis, quotation marks, commas)
  - `TypeError` – operation on incompatible objects (e.g. add together an integer and a string)
  - `NameError` – using a variable before it has a value (e.g. typos in your code)
  - `ValueError` – value incompatible with the operation attempted on it

UNIVERSITY OF LEEDS

Example 1: SyntaxError

```
value_1 = input("Enter the first integer: ")
value_2 = input("Enter the second integer: "
total_value = int(value_1) + int(value_2)
print(total_value)
```

```
Traceback (most recent call last):
File "… syntax_error.py", line 2
    value_2 = input("Enter the second integer: "
                    ^
SyntaxError: '(' was never closed
```

UNIVERSITY OF LEEDS

Example 2: TypeError

```
value_1 = 50
str_message = "Hello"
output = str_message + value_1
print(output)
```

```
Traceback (most recent call last):
  File "… type_error.py", line 4, in <module>
    output = str_message + value_1
             ~~~~~~~~~~~~^~~~~~~~~
TypeError: can only concatenate str (not "int") to str
```

Minimizing Debugging Effort

Modular program design

- Easier to isolate and understand bugs when a program consists of small, single-purpose functions (Single Responsibility Principle)

Thorough unit tests

- Each test represents a specific expectation
- Failures tell you immediately which expectations were not met

Defensive programming

- Use **assertions** to check pre/post-conditions in functions

UNIVERSITY OF LEEDS

Task 1: Common Errors

`week1.7/session2/task1`

1. Run each file in the folder and observe the error message
2. Edit the file to correct the identified error
3. Rerun the file to verify that the error is fixed

UNIVERSITY OF LEEDS

Print-Based Debugging

Just add the appropriate call to a standard print function in places where you need to see information on program status!

Advantages

- Applicable to all languages
- No special tools needed

Disadvantages

- Print statements need to be removed manually from the source code once you have finished debugging

UNIVERSITY OF LEEDS

Enhancement with Logging module

- Python's built-in `logging` module provides utilities for recording debug, info, warning, error, and critical messages
- Instead of using `print()`, logging is more suitable for production environment
  - It's stable
  - Configurable with different logging levels
  - Can be turn on and off

(see https://docs.python.org/3/library/logging.html for details on different logging levels and formatting)

UNIVERSITY OF LEEDS

Using Logging Module

```
import logging

def calculate_average():
    logging.basicConfig(filename="log.txt", filemode="w",
                        level=logging.DEBUG)
    logger = logging.getLogger()
    logger.info("calculate_average(): started")
    total_score = sum(scores)
    total_assignments = len(scores)
    logger.debug(f"{total_score} divided by {total_assignments}")
    …
```

UNIVERSITY OF LEEDS

Task 2: Debugging with `logging` module

`week1.7/session2/task2`

1. Edit `debug_logging.py` to add logging as commented
2. Run the file.
3. Examine the log file to identify the bug.
4. Edit the file to correct the bug.
5. Rerun the file to verify that the bug is fixed.

Assertions

- These are useful for more than unit testing!
- For example, you can put them at the start of a function, to check preconditions such as parameter type and value
- Different from using an `if` statement and raising an exception: assertions can be removed at compile time by using –O command line option with the `python` command

A Better Approach

- Print-based debugging can make code cluttered and hard to read
- A more sophisticated approach is to use a **source-level debugger** to examine the program while it is running
- No changes to source code are needed here!

Source-Level Debugging Tools

A source-level debugger allows us to

- Establish **breakpoints** where program execution is paused
- Step through the program, one statement at a time
- Step into functions when they are called, or execute them without stepping into them, as desired
- Display values of variables, or track changes in their values
- View the **call stack** of the program

Source-Level Debugging Tools

- **Pdb**, a simple Python debugger, is available in the standard library
- IDEs may provide their own source-level debugging

Pdb

See https://docs.python.org/library/pdb.html

Typical usage:

```
python -m pdb prog.py [args]
```

Loads and runs pdb module

Program to be debugged

Code in `prog.py` will be executed under the control of Pdb

Pdb will pause at first line and display the `(Pdb)` prompt, allowing you to input commands

Selected Pdb Commands

Command	Abbr	Purpose
`list`	`l`	List source code, in chunks (start & end lines can be specified)
`break`	`b`	Insert breakpoint at specified line or function, or display all breakpoints
`clear`	`cl`	Clear breakpoint at specified line or function, or clear all breakpoints
`next`	`n`	Execute next line; doesn't step into functions
`step`	`s`	Execute next line, stepping into function if it is a function call
`continue`	`c`	Resume execution (until next breakpoint)
`run` `restart`		(Re)run the program (command line arguments can be provided) Note: cannot be abbreviated to `r`
`p`		Print value of provided expression (e.g., a variable)
`pp`		Pretty-print value of provided expression
`where`	`w`	Display call stack



UNIVERSITY OF LEEDS

Task 3: Debugging with Pdb

`week1.7/session2/task3`

Debugging a simple Python program with Pdb

Follow step-by-step instructions in `README.md`

UNIVERSITY OF LEEDS

Task 4: Debugging with Pdb and unit tests

`week1.7/session2/task4`

Fix a buggy Python program using Pdb and unit tests!

Follow step-by-step instructions in `README.md`

UNIVERSITY OF LEEDS

Summary

We have

- Defined what bugs are and what debugging is
- Looked at print-based debugging & logging
- Considered a better approach: source-level debugging
- Explored basic use of Pdb